

Generador yaml periodos

Víctor Tilve

2025-08-15

Función para generar un archivo yaml que sirve para posteriormente representar en un grafo las estadísticas de medidas de brillo de cielo nocturno:

```
# Necesario
library(yaml)

#' Genera un YAML para run_stats_from_yaml()
#'
#' @param sensors data.frame con columnas:
#'   - sensor_id (obligatoria)
#'   - sensor_label (obligatoria)
#'   - sensor_uri (opcional; si falta, = sensor_id)
#' @param file ruta del YAML a escribir (p.ej. "config.yml")
#'
#' --- MODO 1 (periodo manual, mismo para todos los sensores) ---
#' @param period_start "YYYY-MM-DD HH:MM:SS"
#' @param period_end   "YYYY-MM-DD HH:MM:SS"
#' @param period_label etiqueta manual (obligatoria en modo manual)
#'
#' --- MODO 2 (partir el año completo en N partes) ---
#' @param year   año numérico (p.ej. 2024)
#' @param n_parts entero > 0. Si es 12/6/4/2 se alinea a meses
#'               (Meses/Bimestres/Trimestres/Semestres) con etiquetas automáticas.
#'               En otros casos divide el año en partes iguales por tiempo y usa P1..Pn.
#'
#' --- Parámetros globales que irán a config ---
#' @param base_uri, outfile, mode_method, hist_breaks, digits
#' @param sensor_col, value_col, time_col nombres de columnas en tu DF
#'
generate_yaml_config <- function(
  sensors,
  file = "config.yml",

  # MODO 1 (manual, usa estos tres) - si se usa, NO pases year/n_parts
  period_start = NULL,
  period_end   = NULL,
  period_label = NULL,

  # MODO 2 (auto-partes de un año) - si se usa, NO pases start/end
  year   = NULL,
  n_parts = NULL,

  # Config global
```

```

base_uri    = "http://example.org#",
outfile     = "grafo_estadistica.ttl",
mode_method = "density", # o "hist"
hist_breaks = 30,
digits     = 6,

# Nombres de columnas del DF
sensor_col = "sensor",
value_col  = "valor",
time_col   = "fecha"
) {

  stopifnot(is.data.frame(sensors))
  req_cols <- c("sensor_id", "sensor_label")
  if (!all(req_cols %in% names(sensors))) {
    stop("El data.frame 'sensors' debe tener columnas: sensor_id, sensor_label (y opcional sensor_uri).")
  }

  # Validación de modos
  modo_manual <- !is.null(period_start) && !is.null(period_end)
  modo_auto   <- !is.null(year) && !is.null(n_parts)

  if (modo_manual && modo_auto) {
    stop("Elige UN modo: manual (period_start/period_end/period_label) o auto (year/n_parts).")
  }
  if (!modo_manual && !modo_auto) {
    stop("Debes indicar o bien periodo manual (period_start/period_end/period_label) o bien year/n_parts.")
  }
  if (modo_manual && is.null(period_label)) {
    stop("En el modo manual debes proporcionar 'period_label'.")
  }

  # Helpers de formato fecha/hora
  fmt <- function(dt) format(as.POSIXct(dt, tz = "UTC"), "%Y-%m-%d %H:%M:%S",
                             tz = "UTC")
  make_dt <- function(y, m, d, h=0, mi=0, s=0) {
    as.POSIXct(sprintf("%04d-%02d-%02d %02d:%02d:%02d", y, m, d, h, mi, s),
                tz = "UTC")
  }

  # Genera periodos automáticos alineados a meses cuando n_parts es 12/6/4/2.
  build_periods_auto <- function(year, n_parts) {
    # Casos con alineación por meses:
    if (n_parts %in% c(12, 6, 4, 2)) {
      # Definir grupos de meses
      groups <- switch(as.character(n_parts),
                       "12" = as.list(as.data.frame(matrix(1:12, nrow=12))),
                       "6"  = split(1:12, rep(1:6, each=2)), # bimestres
                       "4"  = split(1:12, rep(1:4, each=3)), # trimestres
                       "2"  = split(1:12, rep(1:2, each=6)))

      # Etiquetas
      labels <- switch(as.character(n_parts),
                       "12" = sprintf("Mes%02d", 1:12, year),

```

```

        "6" = paste0("Bim", 1:6, year),
        "4" = paste0("Trim", 1:4, year),
        "2" = paste0("Sem", 1:2, year))

periods <- vector("list", length(groups))
for (i in seq_along(groups)) {
  ms <- groups[[i]]
  m_start <- min(ms); m_end <- max(ms)
  start <- make_dt(year, m_start, 1, 0, 0, 0)
  # último día del mes final
  first_next <- if (m_end == 12) make_dt(year + 1, 1, 1, 1) else
    make_dt(year, m_end + 1, 1)
  end <- first_next - 1 # 23:59:59 del último día
  periods[[i]] <- list(
    period_start = fmt(start),
    period_end   = fmt(end),
    period_label = labels[i]
  )
}
return(periods)
}

# Caso general: dividir en partes iguales por tiempo
start <- make_dt(year, 1, 1, 0, 0, 0)
end   <- make_dt(year, 12, 31, 23, 59, 59)
dur   <- as.numeric(difftime(end, start, units = "secs"))
cuts  <- start + round(seq(0, dur, length.out = n_parts + 1))

periods <- vector("list", n_parts)
for (i in seq_len(n_parts)) {
  ps <- cuts[i]
  pe <- cuts[i + 1] - 1
  periods[[i]] <- list(
    period_start = fmt(ps),
    period_end   = fmt(pe),
    period_label = paste0("P", i, year) # etiqueta genérica
  )
}
periods
}

# 1) Construir bloque 'config' global
cfg <- list(
  config = list(
    base_uri    = base_uri,
    outfile     = outfile,
    mode_method = mode_method,
    hist_breaks = hist_breaks,
    digits      = digits,
    sensor_col  = sensor_col,
    value_col   = value_col,
    time_col    = time_col
  )
)

```

```

)

# 2) Construir las 'entries'
entries <- list()

if (modo_manual) {
  # Un único periodo para TODOS los sensores
  for (i in seq_len(nrow(sensors))) {
    sid <- sensors$sensor_id[i]
    slbl <- sensors$sensor_label[i]
    suri <- if ("sensor_uri" %in% names(sensors) &&
               !is.na(sensors$sensor_uri[i])) sensors$sensor_uri[i] else sid

    entry <- list(
      sensor_id = as.character(sid),
      sensor_label = as.character(slbl),
      # sensor_uri solo si es distinto de sensor_id
      period_start = fmt(period_start),
      period_end = fmt(period_end),
      period_label = as.character(period_label)
    )
    if (!identical(suri, sid)) entry$sensor_uri <- as.character(suri)

    entries[[length(entries) + 1]] <- entry
  }
} else {
  # MODO AUTO: múltiples periodos por sensor
  periods <- build_periods_auto(year, n_parts)
  for (i in seq_len(nrow(sensors))) {
    sid <- sensors$sensor_id[i]
    slbl <- sensors$sensor_label[i]
    suri <- if ("sensor_uri" %in% names(sensors) &&
               !is.na(sensors$sensor_uri[i])) sensors$sensor_uri[i] else sid

    for (p in periods) {
      entry <- list(
        sensor_id = as.character(sid),
        sensor_label = as.character(slbl),
        period_start = p$period_start,
        period_end = p$period_end,
        period_label = p$period_label
      )
      if (!identical(suri, sid)) entry$sensor_uri <- as.character(suri)

      entries[[length(entries) + 1]] <- entry
    }
  }
}

cfg$entries <- entries

# 3) Escribir YAML
write_yaml(cfg, file)

```

```

message("YAML escrito en: ", normalizePath(file, mustWork = FALSE))
invisible(cfg)
}

```

Aquí se indica la URI (sensor_id) de los fotómetros y el nombre que recibe:

```

sensores <- data.frame(
  sensor_id = c("http://example.org#SQM_Cervantes_ANC_42.820778_-6.922902",
               "http://example.org#SQM_Forcarei_FOR_42.610943_-8.370965"),
  sensor_label = c("SQM_Cervantes", "SQM_Forcarei"),
  # sensor_uri opcional; si no se pone, = sensor_id
  stringsAsFactors = FALSE
)

```

Ejemplo de uso individual. Es decir, así se genera un periodo de forma manual:

```

generate_yaml_config(
  sensors      = sensores,
  file         = "config_manual.yml",
  period_start = "2024-01-01 00:00:00",
  period_end   = "2024-03-31 23:59:59",
  period_label = "Trim12024",
  # Config global (opcional, ya tienen defaults)
  base_uri     = "http://example.org#",
  outfile      = "grafo_estadistica_ejemplo1.ttl",
  mode_method  = "density",
  hist_breaks  = 30,
  digits       = 6,
  sensor_col   = "sensor",
  value_col    = "valor",
  time_col     = "fecha"
)

```

YAML escrito en: /media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering

Ejemplo de generación de un yaml indicando un periodo que se crea automáticamente:

```

generate_yaml_config(
  sensors = sensores,
  file    = "config_auto_trimestres.yml",
  year    = 2024,
  n_parts = 4, # + Trim1..Trim4, alineado a meses
  base_uri = "http://example.org#",
  outfile  = "grafo_estadistica_ejemplo2.ttl"
)

```

YAML escrito en: /media/victor/1FPB-COTARE/Cousas/TFM-IA/TFM-trabajo/00.CousasDefinitivas/clustering